

Punto 3.11 — Composición de consultas y ejecución diferida

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.10. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Composición de consultas y ejecución diferida** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.11
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `BuscarOrdenes`, `Skip`, `Take`, `ToQueryString`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ComposicionConsultasUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ComposicionConsultasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ===");
        var resultado = _unidad.Ordenes.BuscarOrdenes("Constructora del Norte", "Pendiente", new
DateTime(2024,1,1), "fecha", true, 1, 10);
        if (resultado.Elementos.Count != 2) throw new InvalidOperationException("Consulta compuesta
inesperada.");
        Console.WriteLine($"Resultados: {resultado.Elementos.Count}");
        Console.WriteLine(resultado.Sql);
    }
}
```

Explicación línea a línea del caso de uso 3.11

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class ComposicionConsultasUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var resultado = _unidad.Ordenes.BuscarOrdenes("Constructora del Norte", "Pendiente", new DateTime(2024,1,1), "fecha", true, 1, 10);` → Compone filtros opcionales, ordenación y paginación y materializa la consulta una sola vez al final.

Línea 14: `if (resultado.Elementos.Count != 2) throw new InvalidOperationException("Consulta compuesta inesperada.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 15: `Console.WriteLine($"Resultados: {resultado.Elementos.Count}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 16: `Console.WriteLine(resultado.Sql);` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 17: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 18: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

▮ Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ComposicionConsultasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ComposicionConsultasUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.11 OK");

```

Explicación línea a línea de Program.cs 3.11

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa AceriaDbContext para preparar la base de demostración desde el composition root.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar appsettings.json.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<ComposicionConsultasUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<ComposicionConsultasUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.11 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.11 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Skip/Take sin un orden determinista produce páginas inestables; la paginación debe seguir a `OrderBy`.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: APOYO M03 3.11 - PAGINACION SIN ORDEN

```
// Variante deliberadamente incorrecta para demostrar el Paso 9.
ConsultaCompuestaResultadoDto BuscarOrdenesSinOrdenDiagnostics(int pagina, int tamanoPagina);
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: APOYO M03 3.11 - PAGINACION SIN ORDEN

```
// La consulta se pagina sin OrderBy de negocio para mostrar por qué el Paso 9 la considera inestable.
public ConsultaCompuestaResultadoDto BuscarOrdenesSinOrdenDiagnostics(int pagina, int tamanoPagina)
{
    var proyectada = _context.OrdenesFabricacion.AsNoTracking()
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        });

    var paginada = proyectada
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina);

    return new ConsultaCompuestaResultadoDto
    {
        Sql = paginada.ToQueryString(),
        Elementos = paginada.ToList()
    };
}
```

Archivo: src/AceriaData.Application/ComposicionConsultasUseCase.cs

Marcador: ERROR CONTROLADO M03 3.11 - SKIP TAKE SIN ORDEN DETERMINISTA

```
// Demuestra el Paso 9: la paginación puede traducirse, pero sin un OrderBy de negocio no es estable.
var sinOrden = _unidad.Ordenes.BuscarOrdenesSinOrdenDiagnostics(1, 2);
if (sinOrden.Elementos.Count != 2)
    throw new InvalidOperationException("Error controlado 3.11: la página sin orden no devolvió dos filas.");
Console.WriteLine($"Error controlado 3.11 OK | Página sin orden: {sinOrden.Elementos.Count}");
Console.WriteLine("SQL SIN ORDEN INICIO");
Console.WriteLine(sinOrden.Sql);
Console.WriteLine("SQL SIN ORDEN FIN");
```

Comprobación esperada: Skip/Take sin OrderBy explícito no ofrece una paginación determinista.

Laboratorio adicional del punto 3.11

Diagnóstico técnico

La composición flexible permanece dentro de Infrastructure. Filtros opcionales, orden dinámico y paginación se encadenan sin materializar. Skip/Take debe aplicarse tras un orden determinista para que las páginas sean repetibles.

Reto resuelto y verificación adicional

Ejecuta `BuscarOrdenes` con Norte + Pendiente, fecha descendente, página 1 y tamaño 2. El DTO devuelve resultados y `ToQueryString()` de la misma consulta; los filtros ausentes no deben aparecer en SQL.

Errores comunes revisados

Error	Causa	Solución
Materialización prematura	Se llamó a <code>ToList</code> antes de aplicar todos los filtros	Materializar solo al final
Filtros en memoria	Se materializó la consulta antes de filtrar	Construir la consulta con <code>IQueryable</code>
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
ThenBy sin OrderBy	Se llamó a <code>ThenBy</code> sin <code>OrderBy</code> previo	Llamar a <code>OrderBy</code> primero
Skip sin OrderBy	Se aplicó <code>Skip</code> sin ordenar	Aplicar <code>OrderBy</code> antes de <code>Skip</code>

Error	Causa	Solución
Paginación incorrecta	Se calculó mal el Skip	Usar (pagina - 1) * tamañoPagina

Analogía operativa

La composición de consultas en una acería es como construir una orden de búsqueda al archivo central paso a paso. En lugar de pedir todos los documentos y después filtrarlos, el jefe de planta va añadiendo condiciones a la orden: primero el cliente, después el estado, después el rango de fechas. Cada condición se añade a la orden sin enviarla todavía. Solo cuando la orden está completa, el jefe la envía al archivo central. El archivo recibe la orden con todas las condiciones y devuelve solo los documentos que cumplen todas ellas. La ejecución diferida es como preparar la orden sin enviarla: se puede modificar hasta el último momento. La materialización prematura es como enviar la orden antes de terminarla: el archivo devuelve documentos que después hay que filtrar a mano. La composición de consultas permite construir consultas flexibles y eficientes, adaptadas a las necesidades de cada momento.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Añade un filtro opcional y una segunda forma de ordenación sin ejecutar la consulta antes de Skip/Take.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: src/AceriaData.Application/ComposicionConsultasUseCase.cs

Marcador: RETO M03 3.11 - FILTROS OPCIONALES Y SEGUNDA ORDENACION

```
// Demuestra el Paso 10 sin materializar antes de Skip/Take.
var retoFecha = _unidad.Ordenes.BuscarOrdenes(
    "Constructora del Norte", "Pendiente", null, "fecha", true, 1, 2);
var retoCliente = _unidad.Ordenes.BuscarOrdenes(
    null, "Pendiente", null, "cliente", false, 1, 2);

if (retoFecha.Elementos.Count != 2 || retoCliente.Elementos.Count != 2)
    throw new InvalidOperationException("Reto 3.11: cardinalidad inesperada.");

Console.WriteLine("Reto 3.11 OK | Norte+Pendiente fecha desc: 2 | Pendiente cliente asc: 2");
Console.WriteLine("RETO_FECHA_SQL_INICIO");
Console.WriteLine(retoFecha.Sql);
Console.WriteLine("RETO_FECHA_SQL_FIN");
Console.WriteLine("RETO_CLIENTE_SQL_INICIO");
Console.WriteLine(retoCliente.Sql);
Console.WriteLine("RETO_CLIENTE_SQL_FIN");
```

Comprobación esperada: filtros opcionales y dos ordenaciones distintas permanecen diferidos hasta la paginación.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.11 OK, el siguiente estado parte exactamente de esta solución y añade

Buenas prácticas en el acceso a datos y composición de consultas.